Project Design Phase
Problem – Solution Fit Template

Date	05 March2026
Team ID	SWTID-2026-3045
Project Name	SpendWise
Maximum Marks	2 Marks

Problem – Solution Fit Overview:
The Problem–Solution Fit ensures that the identified problem faced by beginners learning backend development is properly addressed by a simple, practical, and hands-on solution. SpendWise focuses on helping beginners understand **API development, JWT authentication, and MongoDB integration** through a small and clear project.

Purpose:

- Address the difficulty beginners face in understanding **backend project structure** and **API workflows**.
- Provide a **simple and practical backend project** to learn CRUD operations.
- Help beginners understand **JWT-based authentication** and protected routes.
- Teach beginners how to **connect and use MongoDB** with Node.js and Express.
- Offer a clean and minimal project that can be tested using Postman or Thunder Client.

Problem Statement:
Many beginners struggle to learn backend development because most examples are either too simple (only basic APIs) or too complex (full production systems). Beginners often find it difficult to understand how **authentication, database connection, and CRUD operations** work together in a real project. This leads to confusion in building structured backend applications and weak practical understanding of REST APIs and security using JWT.

Solution:

- SpendWise is a **simple Node.js, Express, and MongoDB backend project** designed for learning purposes.
- It provides **User Registration and Login** with **JWT authentication**.
- After login, users can perform **CRUD operations** (Create, Read, Update, Delete) on their own data.
- JWT middleware is used to **protect routes**, helping beginners understand API security.
- The project uses a **clean folder structure** (controllers, models, routes, middleware) to teach best practices.
- APIs can be easily tested using **Postman or Thunder Client**, making it beginner-friendly and practical.

Proposed Solution Template:

S.No.	Parameter	Description
1.	Problem Statement (Problem to be solved)	Many beginners find it difficult to understand backend development because examples are either too simple or too complex. They struggle to connect concepts like API creation, JWT authentication, and MongoDB integration into one complete working project.
2.	Idea / Solution description	SpendWise is a simple backend project built using Node.js, Express, and MongoDB that allows users to register, login using JWT, and perform basic CRUD (Create, Read, Update, Delete) operations on their own data. The project provides a clean structure with models, controllers, routes, and middleware to help beginners learn backend development step by step.
3.	Novelty / Uniqueness	The project focuses on learning by doing, combining authentication, database connection, and CRUD operations in one small and easy-to-understand system. It avoids unnecessary complexity and is designed specifically for beginners to practice real-world backend concepts in a simple way.
4.	Social Impact / Customer Satisfaction	SpendWise helps beginners gain practical backend skills, improving their understanding of web development and employability. It provides an accessible learning resource without paid tools and encourages hands-on practice with modern backend technologies.

Solution Architecture:

The solution architecture for the **SpendWise** application ensures a **simple, scalable, and easy-to-understand backend system** for beginners to learn JWT authentication, REST API development, and MongoDB integration. The system is designed using **Node.js, Express, and MongoDB** following a clean and modular structure.

SpendWise follows a **client–server architecture**, where the client (Postman/Thunder Client or optional frontend) sends HTTP requests to the backend API. The backend handles authentication, business logic, and database operations, while MongoDB is used to store user and record data.

Goals of the Solution Architecture:

Identify the Best Tech Solution:

- Use **Node.js and Express.js** to build RESTful APIs.
- Use **MongoDB with Mongoose** for data storage and database operations.
- Use **JWT (JSON Web Token)** for secure user authentication and authorization.
- Use **Postman or Thunder Client** for API testing and demonstration.

Define Structure & Characteristics:

- Follow a **modular project structure** with separate folders for:
  - Controllers (business logic)

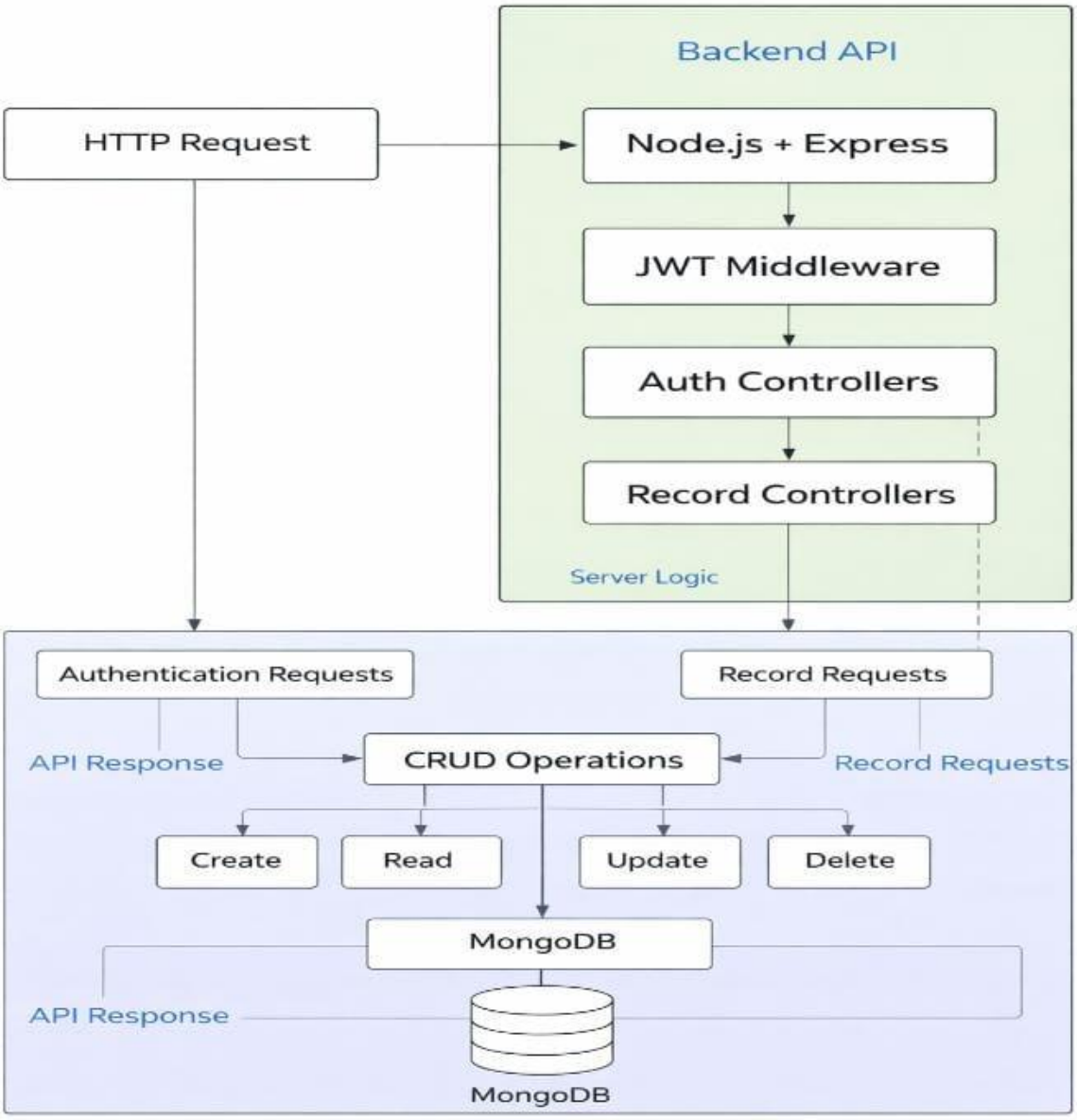
    - Model (database schemas)
    - Routes (API endpoints)
    - Middleware (JWT authentication)
- Ensure the architecture is:
    - **Simple** for beginners to understand
    - **Maintainable** for future changes
    - **Scalable** to add more features later (filters, frontend, etc.)

Outline Features & Development Phases:

- **Phase 1:** Project setup and MongoDB connection
- **Phase 2:** User authentication (Register & Login with JWT)
- **Phase 3:** JWT middleware for protected routes
- **Phase 4:** CRUD operations (Create, Read, Update, Delete) on user data
- **Phase 5:** Testing APIs using Postman/Thunder Client and documentation

Establish Specifications for Development & Delivery:

- The backend must expose **REST APIs** for authentication and CRUD operations.
- All protected routes must require a **valid JWT token**.
- User data and records must be stored securely in **MongoDB**.
- The codebase must follow a **clean and organized structure** for easy learning and maintenance.
- The application should run locally using **Node.js** and be easy to set up using environment variables.


Backend API
HTTP Request
Node.js + Express
JWT Middleware
Auth Controllers
Record Controllers
Server Logic
Authentication Requests
Record Requests
API Response
CRUD Operations
Record Requests
Create
Read
Update
Delete
MongoDB
API Response
MongoDB